Garantie de build, procédure de mise à jour et politique de langue

Date : 2026-10-03 **Sections couvertes :** §II.D Garantie du produit, §II.E Mises à jour, §VI Règles de langue du manuel de certification.

---

1. Garantie d'identité machine-test <-> déploiement client (§II.D)

Question du manuel : "Quelles sont les garanties que vous avez mis en place afin de s'assurer que le logiciel de la machine-test et celui du SCE installé chez le client sont identiques ?"

Réponse :

La correspondance exacte entre le build de certification soumis au SPF Finances et le build déployé chez chaque client est garantie par quatre mécanismes complémentaires :

1.1 Build reproductible

Le dépôt source contient : - `Cargo.lock` commité - fige les versions exactes de toutes les dépendances Rust transitives. - `rust-toolchain.toml` épingle la version exacte du compilateur Rust (canal `1.99.0`). - `package-lock.json` (UI Vue) - fige les versions exactes des dépendances Node.

La build cert est produite par une commande unique et figée, exécutée dans un conteneur Debian 13 `linux/amd64` :

```
./deploy/build-release.sh --docker
# → scripts/build_and_hash.sh : npm ci + build de l'UI, puis
#   cargo build --release --no-default-features --features cert
```

Le conteneur fixe l'environnement de compilation : - l'image Rust est désignée par son empreinte (`rust:1.99.0-trixie@sha256:…`), pas par un tag qui évolue ; sa toolchain est celle de `rust-toolchain.toml` ; - Node.js vient de l'archive officielle nodejs.org, dont l'empreinte SHA-256 est vérifiée avant usage ; - les chemins de l'hôte (dépôt, registre Cargo) sont remplacés dans le binaire par des noms fixes (`--remap-path-prefix`) ; - l'UI embarquée est construite depuis `package-lock.json`.

Deux builds du même commit produisent donc le même SHA-1. La CI le vérifie à chaque modification (`.github/workflows/image.yml`, tâche `repro`, script `scripts/check_repro.sh`) : deux builds complètes depuis un arbre nettoyé, comparaison des SHA-1. Les rapports `build_info_*.txt` des deux builds du tag cert sont joints au dossier (`evidence/build/`).

1.2 Empreinte SHA-1 du binaire

Le script `scripts/build_and_hash.sh` : 1. Compile le build de certification. 2. Vérifie le périmètre du binaire (vérification anti-régression) : commandes en ligne présentes, facturation B2B (`pos_peppol`) absente. 3. Vérifie l'absence de `DELETE FROM fiscal_events` ou `UPDATE fiscal_events` dans le source compilé dans le binaire : tous les fichiers `*.rs` et `*.sql` de `crates/` et de `src-tauri/src/`. Seul le code de test, jamais compilé dans le binaire, en est exclu : les dossiers de tests d'intégration Cargo (le dossier `tests/` à la racine d'une crate, soit `crates/*/tests/`, compilé uniquement par `cargo test`) et les blocs `#[cfg(test)]`. Ces tests exécutent ces requêtes pour prouver que la base les refuse. 4. Calcule SHA-1, SHA-256 et MD5 du binaire. 5. Produit un rapport `build_info_YYYYMMDD-HHMMSS.txt`.

La valeur SHA-1 est déclarée au formulaire §I.D et au tableau §VII.F. Toute installation cliente est vérifiable via :

```
sha1sum /opt/pili-pos/pili-pos
# doit correspondre à la valeur déclarée au SPF
```

1.3 Environnement kiosk scellé

Le premier déploiement se fait par l'image kiosque (clé USB, cf. *MANUEL_INSTALLATION_FR.md* §2-3), construite par `deploy/image/build.sh` (mkosi, qui installe le binaire certifié via `deploy/provision/provision.sh` et vérifie son SHA-1 dans l'image finie) ; les mises à jour remplacent le seul binaire via `deploy/update.sh --sha1` (cf. §2.1), ou le système par l'entrée "Update" de la clé image (*MANUEL_INSTALLATION_FR.md* §6). **La clé USB, sur place, est le seul chemin d'arrivée du binaire fiscal** : jamais par `scp` (aucun serveur SSH n'est installé), jamais pendant une session RustDesk (qui donne accès à l'écran de la caisse, pas à un shell, et ne sert qu'à l'assistance, cf. §2.2).

L'OS hôte (Debian 13 minimal) est configuré lors de l'installation initiale pour constituer un environnement scellé :

- Système de fichiers racine en lecture seule (option `ro` sur la ligne de commande du noyau, drapeau GPT lecture seule sur la partition racine, `deploy/image/mkosi.repart/20-root.conf`).
- Aucun serveur SSH installé (`openssh-server` exclu du paquet de base).
- Aucune mise à jour automatique de paquets (`apt-daily.timer`, `apt-daily-upgrade.timer` et `unattended-upgrades` masqués, `deploy/provision/provision.sh`).
- Seuls le volume `/var` (chiffré LUKS2 : base de données, sauvegardes, journaux et configuration `/etc/pili-pos`) et `/tmp` (en mémoire) sont en écriture.
- Cage Wayland compositor lance directement `pili-pos` en plein écran ; pas de bureau, aucun terminal accessible à l'opérateur : l'écran ne montre que pili-pos. Une console de maintenance locale protégée par le mot de passe `root` existe sur `tty2` (`Ctrl+Alt+F2`, clavier USB branché) ; seul le distributeur s'en sert, sur place.

Une fois l'installation initiale effectuée, toute modification du binaire passe obligatoirement par `deploy/update.sh` avec les droits root.

1.4 Tag git immutable et signé

Le commit du build de certification est tagué `vX.Y.Z` et signé GPG par le producteur. Le SHA git de ce tag est consigné dans le rapport de build et permet à tout moment de reproduire le build à l'identique à partir du dépôt source.

Toute mise à jour ultérieure (correction de bug, nouvelle fonctionnalité) passe par : - Un nouveau tag `vX.Y.Z+1` signé. - Une nouvelle build cert (et donc nouveau SHA-1). - Une nouvelle demande de certification SPF Finances pour la mise à jour (procédure §II.E ci-dessous + Article 109 du Manuel).

---

2. Procédure de mise à jour (§II.E)

Question du manuel : "Pour chaque modification sur le POS, une demande de certification de mise à jour est nécessaire. Ces mises à jour sont testées sur la machine de test. Comment implémentez-vous les mises à jour sur la machine de test ?"

Réponse :

2.1 Mises à jour entre cycles de certification

Aucune mise à jour automatique en arrière-plan n'est implémentée. C'est une décision explicite de scellement de l'appliance fiscale : un POS certifié ne doit pas pouvoir voir son code changé sans une nouvelle validation SPF Finances.

La procédure de mise à jour est la suivante :

1. **Côté producteur :**
   - Implémentation des modifications dans une branche dédiée.
   - Tests automatisés (`cargo test --workspace`) passent à 100%.

- Anti-régression : `cargo build --release --no-default-features --features cert` + `scripts/build_and_hash.sh` (périmètre du binaire : commandes en ligne présentes, `nm pili-pos | grep -c pos_peppol == 0`).
- Nouveau tag git signé vX.Y.Z+1.
- Production du nouvel exécutable Tauri (`pili-pos`) via la commande de build de certification.
- Calcul du SHA-1 du nouveau build.
- Soumission d'une nouvelle demande de certification (mise à jour) au SPF Finances.

2. **Sur la machine de test SPF (pendant le cycle de mise à jour) :**
   - Le producteur remet le nouveau binaire `pili-pos` pour la machine de test sur clé USB, sur place (cf. §2.2).
   - Le binaire est installé via `deploy/update.sh <binaire> --sha1 <SHA-1 déclaré>` : le script refuse tout binaire dont le SHA-1 diffère de la valeur déclarée, sauvegarde binaire et base, applique les migrations puis redémarre le service. Cf. *MANUEL_INSTALLATION_FR.md* §6.1.
   - Tests de validation effectués par l'agent SPF.
3. **Côté client (après acceptation SPF de la mise à jour) :**
   - Mise à jour du binaire client par le distributeur via `deploy/update.sh --sha1`, sur site (jamais par SSH : aucun serveur SSH n'est installé).
   - Sauvegarde préalable de la DB `pos.db` : `deploy/update.sh` en écrit une avant d'installer le binaire (la caisse en écrit aussi une après chaque rapport Z).
   - Vérification post-update du SHA-1.

2.2 Accès à distance

Le manuel recommande l'usage d'une solution d'accès à distance (TeamViewer, AnyDesk, RustDesk, VNC, Zoho Assist, Splashtop) pour faciliter les mises à jour et l'assistance technique.

Position de pili-pos : aucune solution d'accès à distance permanente n'est installée sur la machine de test livrée à l'Intake ni sur la production client. C'est une décision délibérée :

- **Raison principale :** ouvrir un canal d'administration distante sur un terminal de caisse fiscale augmente la surface d'attaque sans bénéfice opérationnel suffisant. Un attaquant qui compromet l'accès distant peut potentiellement contourner les protections fiscales.
- **Raison secondaire :** la spec SCE 2.0 demande un appliance scellé. Un agent SSH ou un client TeamViewer permanent contredit l'esprit du scellement.

Compromis pratique : pour l'assistance et le diagnostic d'un problème reporté par le client (jamais pour installer ou remplacer un binaire, qui arrive toujours sur place par clé USB) : - Client : RustDesk portable (open source, AppImage autonome), version épinglée et vérifiée par SHA-256 à la construction de l'image, extrait dans `/opt/pili-pos/rustdesk/` (lancé via `/opt/pili-pos/rustdesk/AppRun`). Il fait partie de l'image scellée (donc couvert par la garantie d'identité machine de test / production, §1) mais **n'est jamais lancé au boot** et n'installe aucun service. - Ouverture : depuis la caisse, par un Gérant connecté qui confirme avec son PIN ("Config" > "Assistance à distance" > "Démarrer", `crates/pos-engine/src/commands/support.rs`). RustDesk affiche un identifiant et un mot de passe à usage unique, communiqués au producteur par téléphone. - Pendant la session, chaque écran de la caisse, écran de connexion compris, affiche le bandeau "Assistance à distance en cours" ; la session continue après une déconnexion. Démarrage et arrêt sont journalisés (qui, quand). - La session prend fin par "Arrêter la session" (dans "Config", ou sur le bandeau de l'écran de connexion, par n'importe qui), à la fermeture de RustDesk, au redémarrage du terminal, ou au plus tard 2 heures après le démarrage. Une seule session à la fois. Aucun accès permanent, aucun port en écoute hors session. - La session donne accès à l'écran de la caisse, pas à un shell : les mises à jour du binaire se font sur place (clé USB, `deploy/update.sh --sha1`). La saisie à distance (clavier, tactile) sous le compositeur kiosque (Cage) n'est pas garantie : à défaut, la session reste en lecture seule et le producteur guide le Gérant par téléphone. - Pour la machine de test, le SPF Finances peut demander qu'une session de test soit ouverte de la même façon et ses identifiants fournis (manuel §II.E).

Au rendez-vous d'admission SPF Finances :

- Accès console physique direct par l'agent (clavier USB branché pour les vérifications).
- Compte `spf` pré-créé avec mot de passe communiqué sur place.

- Aucun besoin d'accès à distance pendant l'Intake.

---

3. Politique de langue des tickets (§VI)

Question du manuel : règles de langue selon §VI - le ticket doit être disponible en français, néerlandais, allemand selon le besoin (anglais en zones touristiques).

Réponse :

3.1 Langues supportées par le code source

Le code de `pili-pos` supporte le rendu multilingue des tickets via deux mécanismes complémentaires :

- **Libellés produits** : table `translations` polymorphe (cf. *SCHEMA_BD_FR.md*) - couvre catégories, articles, modificateurs. Les lignes du ticket portent le libellé dénormalisé dans la langue active au moment de la vente.
- **Mentions fiscales imprimées** : jeux de libellés FR/NL/DE embarqués dans le code (`crates/pos-fiscal/src/ticket.rs` : `ReceiptLanguage`, `ticket_type_label`, `invalid_disclaimer` ; `crates/pos-print/src/labels.rs` : libellés fixes Date/Ticket/TOTAL/TVA du reçu et du rapport, noms d'événement N/P/T/C/I/F/S/R, noms de moyen de paiement). Les rapports X et Z, le rapport utilisateur (M182/M183/M184) et les noms de moyen de paiement imprimés (Especes/Contant/Bar, Carte/Kaart/Karte) suivent la même langue que le reçu : ce n'est pas propre au ticket de vente. La langue est déterminée par la clé `language` du fichier de configuration `/etc/pili-pos/pos.json`, mappée sur une langue nationale : `fr`/`nl`/`de` directement, toute autre valeur (en, zh) retombe sur le FR.

Langues des mentions fiscales supportées par le code : - `fr` (français) - **complète** (libellés issus de la description détaillée POS v2.2 et du manuel §VI) - `nl` (néerlandais) - **complète** (libellés issus des circulaires GKS) - `de` (allemand) - **complète** (traduction de style officiel, en ASCII ; les libellés allemands ne sont pas tirés d'une version allemande de la spécification, qui ne fait pas partie des documents de référence)

Langues d'affichage de l'interface (indépendantes du ticket) : `fr`, `en`, `zh`.

3.2 Règle appliquée : langue de la région linguistique de l'établissement

Le manuel (§VI, p. 99) dispose : *« En Belgique, le ticket doit être émis en français, en néerlandais ou en allemand, selon la langue officielle de la région linguistique où il est émis. Dans les lieux considérés comme des zones touristiques, le ticket peut également être émis en anglais. »*

La langue des mentions fiscales est donc fixée **par installation**, lors de la mise en service, d'après l'adresse de l'établissement où le POS est exploité (et non d'après le siège du producteur ni la langue de la clientèle) :

Région linguistique de l'établissement	Valeur de `pos.json::language`
Région de langue française	`fr`
Région de langue néerlandaise	`nl`
Région de langue allemande	`de`
Région bilingue de Bruxelles-Capitale	`fr` ou `nl` (les deux langues sont officielles)

L'anglais (zones touristiques) n'est pas proposé comme langue de ticket : toute valeur autre que `fr`/`nl`/`de` retombe sur le FR (cf. §3.1).

Le paramètre fait partie de la checklist de mise en service (`MANUEL_INSTALLATION_FR.md`) et figure sur chaque installation déclarée dans `LISTE_DISTRIBUTEURS_FR.md`.

Exemple : un établissement situé à 4000 Liège se trouve en **région de langue française** → `language = "fr"`, tickets imprimés en français.

Pour basculer la langue des mentions fiscales (FR → NL ou DE), par exemple pour une installation dans une autre région linguistique, la procédure est une mise à jour de configuration (pas du code) : 1. Compléter les traductions manquantes dans la table `translations` pour les articles. 2. Éditer la clé `language` de `/etc/pili-pos/pos.json` (`fr` / `nl` / `de`). 3. Redémarrer le service `pili-pos` (la configuration est lue au boot).

Les trois jeux de mentions FR/NL/DE étant déjà compilés dans le binaire, aucune modification du binaire `pili-pos` n'est nécessaire pour changer de langue : c'est de la **configuration**, pas du code. Donc pas de nouveau cycle de certification.

3.3 Tickets multilingues pour la machine de test SPF

Pour la machine de test livrée à l'Intake, le seed `pos.db` sera configuré avec les traductions FR/NL/DE/EN des produits du manuel (Dry Martini, Burger of the Chef, etc.). L'agent SPF pourra basculer la langue des mentions fiscales en éditant la clé `language` de `/etc/pili-pos/pos.json` (procédure §3.2) et constater le changement sur le ticket suivant.

---

4. Récapitulatif des engagements

Engagement	Mécanisme	Vérifiabilité
Build cert reproductible	`Cargo.lock`, `rust-toolchain.toml`, image de build par empreinte, Node.js par SHA-256	`scripts/check_repro.sh` : deux builds, même SHA-1 (CI, tâche `repro`)
Build = déploiement client	Binaire Tauri scellé + SHA-1 vérifiable	`sha1sum /opt/pili-pos/pili-pos`
Pas de modification post-cert	Système de fichiers racine monté en lecture seule (option `ro` du noyau, drapeau GPT lecture seule) ; binaire identifié par son SHA-1	`sha1sum` du binaire installé comparé au SHA-1 déclaré ; `pili-pos --version`
Nouvelle MAJ = nouveau cert	Procédure §II.E + Article 109 du manuel	Chaque tag vX.Y.Z+1 ouvre une nouvelle demande SPF
Pas d'accès distant permanent	Pas de SSH ; RustDesk portable présent mais jamais lancé au boot	Vérification au scan réseau du POS (aucun port ouvert hors session)
Ticket dans la langue de la région linguistique de l'établissement	`translations` (libellés produits) + `pos.json::language` (mentions fiscales FR/NL/DE)	Inspection visuelle du ticket, paramétrage modifiable sans changer le binaire